**NAME: K. Harshavardhan Reddy**
**REG NO: 192312677L**

# EXPERIMENT NO – 2

## Title:

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Candidate-Elimination algorithm in python to output a description of the set of all hypotheses consistent with the training examples

## Experiment 1.1: Enjoy Sport

### Aim

To read the EnjoySport training examples from a CSV file and determine the version-space boundary hypotheses.

### Procedure

- Save the experiment's input table as its CSV file.
- Set CSV_FILE and TARGET_COLUMN in the Python program to the corresponding file name and target column.
- Initialize S with the most specific hypothesis and G with the most general hypothesis.
- For each positive example, remove inconsistent hypotheses from G and minimally generalize S.
- For each negative example, remove inconsistent hypotheses from S and minimally specialize G.
- Print S and G; hypotheses between the boundaries form the version space.

### Code

```
import csv

CSV_FILE = "candidate_elimination_experiment_1.csv"
TARGET_COLUMN = "EnjoySport"
POSITIVE = "Yes"

def covers(h, x):
    return all(a == "?" or a == b for a, b in zip(h, x))

def geq(h1, h2):
    """True if h1 is at least as general as h2."""
    return all(a == "?" or a == b for a, b in zip(h1, h2))
```

```
def prune_S(S):
    return [h for h in S if not any(
        h != other and geq(h, other) for other in S
    )]

def prune_G(G):
    return [h for h in G if not any(
        h != other and geq(other, h) for other in G
    )]

with open(CSV_FILE, newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    records = list(reader)
    attributes = [c for c in reader.fieldnames if c != TARGET_COLUMN]

X = [[r[a].strip() for a in attributes] for r in records]
y = [r[TARGET_COLUMN].strip() for r in records]
domains = [sorted({x[i] for x in X}) for i in range(len(attributes))]
n = len(attributes)
S = [["Ø"] * n]
G = [["?"] * n]

for x, label in zip(X, y):
    if label == POSITIVE:
        G = [g for g in G if covers(g, x)]
        new_S = []
        for s in S:
            if s == ["Ø"] * n:
                new_S.append(x.copy())
            else:
                h = s.copy()
                for i in range(n):
                    if h[i] != x[i]:
                        h[i] = "?"
                new_S.append(h)
        S = prune_S(new_S)
    else:
        S = [s for s in S if not covers(s, x)]
        new_G = []
        for g in G:
            if covers(g, x):
```

```
            for i in range(n):
                if g[i] == "?":
                    for value in domains[i]:
                        if value != x[i]:
                            h = g.copy()
                            h[i] = value
                            if any(geq(h, s) for s in S):
                                new_G.append(h)
                else:
                    new_G.append(g.copy())
        unique = []
        for h in new_G:
            if h not in unique:
                unique.append(h)
        G = prune_G(unique)

print("Attributes:", attributes)
print("Specific boundary S:", S)
print("General boundary G:", G)
```

## Input

CSV file: candidate_elimination_experiment_1.csv

| Sky | AirTemp | Humidity | Wind | Water | Forecast | EnjoySport |
|---|---|---|---|---|---|---|
| Sunny | Warm | Normal | Strong | Warm | Same | Yes |
| Sunny | Warm | High | Strong | Warm | Same | Yes |
| Rainy | Cold | High | Strong | Warm | Change | No |
| Sunny | Warm | High | Strong | Cool | Change | Yes |

## Output

Expected final boundaries for this illustrative dataset:

Specific boundary S: [['Sunny', 'Warm', '?', 'Strong', '?', '?']]

General boundary G: [['Sunny', '?', '?', '?', '?', '?'], ['?', 'Warm', '?', '?', '?', '?']]

## Result

Candidate-Elimination was demonstrated with this experiment's own CSV input. S represents the maximally specific surviving hypothesis boundary and G represents the maximally general surviving boundary. The version space consists of hypotheses between these boundaries.

## Experiment 1.2: Scholarship Eligibility

### Title

Implement Candidate-Elimination using a CSV training dataset to output the S and G boundaries consistent with the training examples.

### Aim

To read the Eligible training examples from a CSV file and determine the version-space boundary hypotheses.

### Procedure

- Save the experiment's input table as its CSV file.
- Set CSV_FILE and TARGET_COLUMN in the Python program to the corresponding file name and target column.
- Initialize S with the most specific hypothesis and G with the most general hypothesis.
- For each positive example, remove inconsistent hypotheses from G and minimally generalize S.
- For each negative example, remove inconsistent hypotheses from S and minimally specialize G.
- Print S and G; hypotheses between the boundaries form the version space.

### Code

```
import csv

CSV_FILE = "candidate_elimination_experiment_2.csv"
TARGET_COLUMN = "Eligible"
POSITIVE = "Yes"

def covers(h, x):
    return all(a == "?" or a == b for a, b in zip(h, x))

def geq(h1, h2):
    """True if h1 is at least as general as h2."""
    return all(a == "?" or a == b for a, b in zip(h1, h2))

def prune_S(S):
    return [h for h in S if not any(
        h != other and geq(h, other) for other in S
    )]

def prune_G(G):
    return [h for h in G if not any(
        h != other and geq(other, h) for other in G
    )]
```

```
with open(CSV_FILE, newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    records = list(reader)
    attributes = [c for c in reader.fieldnames if c != TARGET_COLUMN]

X = [[r[a].strip() for a in attributes] for r in records]
y = [r[TARGET_COLUMN].strip() for r in records]
domains = [sorted({x[i] for x in X}) for i in range(len(attributes))]
n = len(attributes)
S = [["Ø"] * n]
G = [["?"] * n]

for x, label in zip(X, y):
    if label == POSITIVE:
        G = [g for g in G if covers(g, x)]
        new_S = []
        for s in S:
            if s == ["Ø"] * n:
                new_S.append(x.copy())
            else:
                h = s.copy()
                for i in range(n):
                    if h[i] != x[i]:
                        h[i] = "?"
                new_S.append(h)
        S = prune_S(new_S)
    else:
        S = [s for s in S if not covers(s, x)]
        new_G = []
        for g in G:
            if covers(g, x):
                for i in range(n):
                    if g[i] == "?":
                        for value in domains[i]:
                            if value != x[i]:
                                h = g.copy()
                                h[i] = value
                                if any(geq(h, s) for s in S):
                                    new_G.append(h)
            else:
```

```
            new_G.append(g.copy())
        unique = []
        for h in new_G:
            if h not in unique:
                unique.append(h)
        G = prune_G(unique)

print("Attributes:", attributes)
print("Specific boundary S:", S)
print("General boundary G:", G)
```

## Input

CSV file: candidate_elimination_experiment_2.csv

| Marks | Attendance | Projects | Sports | Income | Eligible |
|---|---|---|---|---|---|
| High | Good | Completed | Yes | Low | Yes |
| High | Good | Completed | No | Low | Yes |
| Medium | Poor | Pending | No | High | No |
| High | Good | Completed | Yes | Medium | Yes |

## Output

Expected final boundaries for this illustrative dataset:

Specific boundary S: [['High', 'Good', 'Completed', '?', '?']]

General boundary G: [['High', '?', '?', '?', '?'], ['?', 'Good', '?', '?', '?'], ['?', '?', 'Completed', '?', '?']]

## Result

Candidate-Elimination was demonstrated with this experiment's own CSV input. S represents the maximally specific surviving hypothesis boundary and G represents the maximally general surviving boundary. The version space consists of hypotheses between these boundaries.

## Experiment 1.3: Play Cricket

### Title

Implement Candidate-Elimination using a CSV training dataset to output the S and G boundaries consistent with the training examples.

### Aim

To read the PlayCricket training examples from a CSV file and determine the version-space boundary hypotheses.

### Procedure

- Save the experiment's input table as its CSV file.
- Set CSV_FILE and TARGET_COLUMN in the Python program to the corresponding file name and target column.
- Initialize S with the most specific hypothesis and G with the most general hypothesis.
- For each positive example, remove inconsistent hypotheses from G and minimally generalize S.
- For each negative example, remove inconsistent hypotheses from S and minimally specialize G.
- Print S and G; hypotheses between the boundaries form the version space.

### Code

```
import csv

CSV_FILE = "candidate_elimination_experiment_3.csv"
TARGET_COLUMN = "PlayCricket"
POSITIVE = "Yes"

def covers(h, x):
    return all(a == "?" or a == b for a, b in zip(h, x))

def geq(h1, h2):
    """True if h1 is at least as general as h2."""
    return all(a == "?" or a == b for a, b in zip(h1, h2))

def prune_S(S):
    return [h for h in S if not any(
        h != other and geq(h, other) for other in S
    )]

def prune_G(G):
    return [h for h in G if not any(
        h != other and geq(other, h) for other in G
    )]
```

```
with open(CSV_FILE, newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    records = list(reader)
    attributes = [c for c in reader.fieldnames if c != TARGET_COLUMN]

X = [[r[a].strip() for a in attributes] for r in records]
y = [r[TARGET_COLUMN].strip() for r in records]
domains = [sorted({x[i] for x in X}) for i in range(len(attributes))]
n = len(attributes)
S = [["Ø"] * n]
G = [["?"] * n]

for x, label in zip(X, y):
    if label == POSITIVE:
        G = [g for g in G if covers(g, x)]
        new_S = []
        for s in S:
            if s == ["Ø"] * n:
                new_S.append(x.copy())
            else:
                h = s.copy()
                for i in range(n):
                    if h[i] != x[i]:
                        h[i] = "?"
                new_S.append(h)
        S = prune_S(new_S)
    else:
        S = [s for s in S if not covers(s, x)]
        new_G = []
        for g in G:
            if covers(g, x):
                for i in range(n):
                    if g[i] == "?":
                        for value in domains[i]:
                            if value != x[i]:
                                h = g.copy()
                                h[i] = value
                                if any(geq(h, s) for s in S):
                                    new_G.append(h)
            else:
```

```
            new_G.append(g.copy())
        unique = []
        for h in new_G:
            if h not in unique:
                unique.append(h)
        G = prune_G(unique)

print("Attributes:", attributes)
print("Specific boundary S:", S)
print("General boundary G:", G)
```

## Input

CSV file: candidate_elimination_experiment_3.csv

| Weather | Temperature | Ground | Wind | Visibility | PlayCricket |
|---|---|---|---|---|---|
| Clear | Mild | Dry | Low | Good | Yes |
| Clear | Warm | Dry | Low | Good | Yes |
| Rainy | Cool | Wet | High | Poor | No |
| Clear | Warm | Dry | High | Good | Yes |

## Output

Expected final boundaries for this illustrative dataset:

Specific boundary S: [['Clear', '?', 'Dry', '?', 'Good']]

General boundary G: [['Clear', '?', '?', '?', '?'], ['?', '?', 'Dry', '?', '?'], ['?', '?', '?', '?', 'Good']]

## Result

Candidate-Elimination was demonstrated with this experiment's own CSV input. S represents the maximally specific surviving hypothesis boundary and G represents the maximally general surviving boundary. The version space consists of hypotheses between these boundaries.

## Experiment 1.4: Loan Approval

### Title

Implement Candidate-Elimination using a CSV training dataset to output the S and G boundaries consistent with the training examples.

### Aim

To read the LoanApproved training examples from a CSV file and determine the version-space boundary hypotheses.

### Procedure

- Save the experiment's input table as its CSV file.
- Set CSV_FILE and TARGET_COLUMN in the Python program to the corresponding file name and target column.
- Initialize S with the most specific hypothesis and G with the most general hypothesis.
- For each positive example, remove inconsistent hypotheses from G and minimally generalize S.
- For each negative example, remove inconsistent hypotheses from S and minimally specialize G.
- Print S and G; hypotheses between the boundaries form the version space.

### Code

```
import csv

CSV_FILE = "candidate_elimination_experiment_4.csv"
TARGET_COLUMN = "LoanApproved"
POSITIVE = "Yes"

def covers(h, x):
    return all(a == "?" or a == b for a, b in zip(h, x))

def geq(h1, h2):
    """True if h1 is at least as general as h2."""
    return all(a == "?" or a == b for a, b in zip(h1, h2))

def prune_S(S):
    return [h for h in S if not any(
        h != other and geq(h, other) for other in S
    )]

def prune_G(G):
    return [h for h in G if not any(
        h != other and geq(other, h) for other in G
    )]
```

```
with open(CSV_FILE, newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    records = list(reader)
    attributes = [c for c in reader.fieldnames if c != TARGET_COLUMN]

X = [[r[a].strip() for a in attributes] for r in records]
y = [r[TARGET_COLUMN].strip() for r in records]
domains = [sorted({x[i] for x in X}) for i in range(len(attributes))]
n = len(attributes)
S = [["Ø"] * n]
G = [["?"] * n]

for x, label in zip(X, y):
    if label == POSITIVE:
        G = [g for g in G if covers(g, x)]
        new_S = []
        for s in S:
            if s == ["Ø"] * n:
                new_S.append(x.copy())
            else:
                h = s.copy()
                for i in range(n):
                    if h[i] != x[i]:
                        h[i] = "?"
                new_S.append(h)
        S = prune_S(new_S)
    else:
        S = [s for s in S if not covers(s, x)]
        new_G = []
        for g in G:
            if covers(g, x):
                for i in range(n):
                    if g[i] == "?":
                        for value in domains[i]:
                            if value != x[i]:
                                h = g.copy()
                                h[i] = value
                                if any(geq(h, s) for s in S):
                                    new_G.append(h)
            else:
```

```
            new_G.append(g.copy())
        unique = []
        for h in new_G:
            if h not in unique:
                unique.append(h)
        G = prune_G(unique)

print("Attributes:", attributes)
print("Specific boundary S:", S)
print("General boundary G:", G)
```

## Input

CSV file: candidate_elimination_experiment_4.csv

| CreditScore | Employment | Income | Debt | Documents | LoanApproved |
|---|---|---|---|---|---|
| Good | Permanent | High | Low | Complete | Yes |
| Good | Permanent | Medium | Low | Complete | Yes |
| Poor | Temporary | Low | High | Incomplete | No |
| Good | Permanent | High | Medium | Complete | Yes |

## Output

Expected final boundaries for this illustrative dataset:

Specific boundary S: [['Good', 'Permanent', '?', '?', 'Complete']]

General boundary G: [['Good', '?', '?', '?', '?'], ['?', 'Permanent', '?', '?', '?'], ['?', '?', '?', '?', 'Complete']]

## Result

Candidate-Elimination was demonstrated with this experiment's own CSV input. S represents the maximally specific surviving hypothesis boundary and G represents the maximally general surviving boundary. The version space consists of hypotheses between these boundaries.

## Experiment 1.5: Suitable Plant Growth

### Title

Implement Candidate-Elimination using a CSV training dataset to output the S and G boundaries consistent with the training examples.

### Aim

To read the GoodGrowth training examples from a CSV file and determine the version-space boundary hypotheses.

### Procedure

- Save the experiment's input table as its CSV file.
- Set CSV_FILE and TARGET_COLUMN in the Python program to the corresponding file name and target column.
- Initialize S with the most specific hypothesis and G with the most general hypothesis.
- For each positive example, remove inconsistent hypotheses from G and minimally generalize S.
- For each negative example, remove inconsistent hypotheses from S and minimally specialize G.
- Print S and G; hypotheses between the boundaries form the version space.

### Code

```
import csv

CSV_FILE = "candidate_elimination_experiment_5.csv"
TARGET_COLUMN = "GoodGrowth"
POSITIVE = "Yes"

def covers(h, x):
    return all(a == "?" or a == b for a, b in zip(h, x))

def geq(h1, h2):
    """True if h1 is at least as general as h2."""
    return all(a == "?" or a == b for a, b in zip(h1, h2))

def prune_S(S):
    return [h for h in S if not any(
        h != other and geq(h, other) for other in S
    )]

def prune_G(G):
    return [h for h in G if not any(
        h != other and geq(other, h) for other in G
    )]
```

```
with open(CSV_FILE, newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    records = list(reader)
    attributes = [c for c in reader.fieldnames if c != TARGET_COLUMN]

X = [[r[a].strip() for a in attributes] for r in records]
y = [r[TARGET_COLUMN].strip() for r in records]
domains = [sorted({x[i] for x in X}) for i in range(len(attributes))]
n = len(attributes)
S = [["Ø"] * n]
G = [["?"] * n]

for x, label in zip(X, y):
    if label == POSITIVE:
        G = [g for g in G if covers(g, x)]
        new_S = []
        for s in S:
            if s == ["Ø"] * n:
                new_S.append(x.copy())
            else:
                h = s.copy()
                for i in range(n):
                    if h[i] != x[i]:
                        h[i] = "?"
                new_S.append(h)
        S = prune_S(new_S)
    else:
        S = [s for s in S if not covers(s, x)]
        new_G = []
        for g in G:
            if covers(g, x):
                for i in range(n):
                    if g[i] == "?":
                        for value in domains[i]:
                            if value != x[i]:
                                h = g.copy()
                                h[i] = value
                                if any(geq(h, s) for s in S):
                                    new_G.append(h)
            else:
```

```
            new_G.append(g.copy())
        unique = []
        for h in new_G:
            if h not in unique:
                unique.append(h)
        G = prune_G(unique)

print("Attributes:", attributes)
print("Specific boundary S:", S)
print("General boundary G:", G)
```

## Input

CSV file: candidate_elimination_experiment_5.csv

| Soil | Sunlight | Water | Temperature | Fertilizer | GoodGrowth |
|---|---|---|---|---|---|
| Loamy | High | Moderate | Warm | Organic | Yes |
| Loamy | High | Moderate | Mild | Organic | Yes |
| Sandy | Low | Low | Cold | None | No |
| Loamy | High | High | Warm | Organic | Yes |

## Output

Expected final boundaries for this illustrative dataset:

Specific boundary S: [['Loamy', 'High', '?', '?', 'Organic']]

General boundary G: [['Loamy', '?', '?', '?', '?'], ['?', 'High', '?', '?', '?'], ['?', '?', '?', '?', 'Organic']]

## Result

Candidate-Elimination was demonstrated with this experiment's own CSV input. S represents the maximally specific surviving hypothesis boundary and G represents the maximally general surviving boundary. The version space consists of hypotheses between these boundaries.